

NLP__Practical__03

April 7, 2026

0.1 Step-1:Import Libraries

```
import pandas as pd
import numpy as np
import re

import nltk
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')

from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

from sklearn.preprocessing import LabelEncoder
from sklearn.feature_extraction.text import TfidfVectorizer
```

0.2 Step-2: Sample Database

```
[2]: data = {
    "text": [
        "I love Machine Learning!!!",
        "Python is GREAT for Data Science.",
        "AI is the future??",
        "I am learning NLP & AI :)"
    ],
    "label": ["positive", "positive", "neutral", "positive"]
}

df = pd.DataFrame(data)
print("Original Data:\n", df)
```

Original Data:

	text	label
0	I love Machine Learning!!!	positive
1	Python is GREAT for Data Science.	positive
2	AI is the future??	neutral
3	I am learning NLP & AI :)	positive

0.3 Step-3: Perform all method

0.3.1 1. Text Cleaning

```
[4]: def clean_text(text):  
      text = text.lower() # lowercase  
      text = re.sub(r'[^a-z\s]', '', text) # remove punctuation & numbers  
      return text  
  
df['cleaned_text'] = df['text'].apply(clean_text)
```

0.3.2 2. Tokenization

```
[5]: df['tokens'] = df['cleaned_text'].apply(word_tokenize)
```

0.3.3 3. Stopword Removal

```
[9]: stop_words = set(stopwords.words('english'))  
  
df['no_stopwords'] = df['tokens'].apply(lambda words: [w for w in words if w  
    ↪ not in stop_words])  
  
# Join words back to sentence  
df['final_text'] = df['no_stopwords'].apply(lambda words: " ".join(words))  
  
print("\nProcessed Data:\n", df[['text', 'final_text']])
```

Processed Data:

	text	final_text
0	I love Machine Learning!!!	love machine learning
1	Python is GREAT for Data Science.	python great data science
2	AI is the future??	ai future
3	I am learning NLP & AI :)	learning nlp ai

0.3.4 4. Lemmatization

```
[8]: lemmatizer = WordNetLemmatizer()  
  
df['lemmatized'] = df['no_stopwords'].apply(lambda words: [lemmatizer.  
    ↪ lemmatize(w) for w in words])
```

0.3.5 5. Label Encoding

```
[10]: le = LabelEncoder()  
df['label_encoded'] = le.fit_transform(df['label'])  
  
print("\nEncoded Labels:\n", df[['label', 'label_encoded']])
```

Encoded Labels:

	label	label_encoded
0	positive	1
1	positive	1
2	neutral	0
3	positive	1

0.3.6 6. TF-IDF

```
[11]: tfidf = TfidfVectorizer()
X_tfidf = tfidf.fit_transform(df['final_text'])

tfidf_df = pd.DataFrame(X_tfidf.toarray(), columns=tfidf.
    ↳get_feature_names_out())

print("\nTF-IDF Representation:\n", tfidf_df)
```

TF-IDF Representation:

	ai	data	future	great	learning	love	machine	nlp	\
0	0.000000	0.0	0.000000	0.0	0.486934	0.617614	0.617614	0.000000	
1	0.000000	0.5	0.000000	0.5	0.000000	0.000000	0.000000	0.000000	
2	0.619130	0.0	0.785288	0.0	0.000000	0.000000	0.000000	0.000000	
3	0.526405	0.0	0.000000	0.0	0.526405	0.000000	0.000000	0.667679	

	python	science
0	0.0	0.0
1	0.5	0.5
2	0.0	0.0
3	0.0	0.0

0.3.7 7. Save Outputs

```
[13]: # Save processed dataset
df.to_csv("processed_text.csv", index=False)

# Save TF-IDF matrix
tfidf_df.to_csv("tfidf_output.csv", index=False)

print("\nFiles saved successfully!")
```

Files saved successfully!

```
[ ]:
```